GECT CONNECT Campus Messaging Application

Team 1 — User Registration & Authentication Module

Technical Documentation

Government Engineering College Thrissur (GEC Thrissur)

Academic Year 2024 – 25

No.	Name	Role	Responsibilities
1	**Maria Shibi**	Team Lead	Project coordination,Integration
2	**Libiya Sebu**	Developer	Login Screen, Session Management, User DAO
3	**Jishnu S**	Developer	Dashboard, AUTH Service, DB Connection
4	**Jayasurya Madhav Jayan**	Developer	Registration Screen, Forgot Password Verification

1. Introduction

GECT Connect is a campus-specific chat messenger application designed exclusively for the students and staff of Government Engineering College Thrissur (GEC Thrissur). Functioning as a WhatsApp-like messenger tailored for the college community, it enables seamless communication through individual chats, group chats, academic notifications, media sharing, and a campus events feed.

The project is divided among 7 teams, each responsible for a specific functional module. Team 1 is assigned the User Registration and Authentication Module — the foundational component on which all other modules depend.

1.1 Project Objectives

- Provide a secure, college-email-restricted registration and login system.
- Distinguish between Student and Staff user roles with role-specific data fields.
- Simulate OTP-based verification for registration and password recovery.
- Maintain clean layered architecture (UI → Service → DAO → Model).
- Demonstrate OOP principles: inheritance, encapsulation, constructor overloading, method overriding.

1.2 Scope of Team 1

Team 1 is responsible for the following functional areas:

- User registration with college email validation (@gectcr.ac.in).
- Role selection (Student / Staff) with role-specific identifier fields.
- OTP simulation for email verification during registration.
- User login with credential validation.
- Forgot-password flow with OTP and password reset.
- Post-login dashboard shell exposing session data to other modules.

1.3 Definitions and Abbreviations

Term	Definition
GECT	Government Engineering College Thrissur
OTP	One-Time Password — a simulated 6-digit code used for verification
JDBC	Java Database Connectivity — API used for MySQL database operations
DAO	Data Access Object — design pattern for database operations
GUI	Graphical User Interface — implemented using Java Swing
OOP	Object-Oriented Programming
FK	Foreign Key — a database constraint linking tables

2.1 Product Perspective

GECT Connect is a standalone desktop application built using Java Swing for the frontend and MySQL for the backend. The authentication module is the foundation layer upon which all other six modules are built. It exposes shared components — DBConnection, SessionManager, and ScreenNavigator — that every other team imports and uses directly.

2.2 Product Functions — Summary

No.	Function	Description
F01	**User Registration**	Allows new students and staff to create accounts using their college email
F02	**Login Authentication**	Verifies credentials and initiates a session for valid users
F03	**OTP Verification**	Simulates OTP-based email verification for account activation
F04	**Forgot Password**	Three-step flow to reset password via OTP verification
F05	**Dashboard Navigation**	Post-login hub screen with navigation to all 7 module areas
F06	**Session Management**	Stores and provides the logged-in user object throughout the app lifetime
F07	**Input Validation**	Validates email format, password strength, mobile number, and empty fields

2.3 User Classes

The system serves two distinct user classes:

Attribute	Student	Staff / Faculty
Role Value	STUDENT	STAFF
Unique Field	Roll Number	Employee ID
Email Domain	@gectcr.ac.in	@gectcr.ac.in
Can Post Announcements	No	Yes

2.4 Operating Environment

- Operating System: Windows 10 / Windows 11
- Java Version: JDK 17 (Temurin / Adoptium)
- Database: MySQL Community Server 8.0

- GUI Framework: Java Swing (built into JDK)
- Database Connectivity: JDBC with MySQL Connector/J 9.6.0
- IDE / Editor: Visual Studio Code with Extension Pack for Java

2.5 Constraints

- Only college email addresses ending with @gectcr.ac.in are accepted for registration
- OTP is simulated — no real email is sent; the generated code is shown in a dialog box
- Passwords are stored as plain text in the current version (hashing to be added in production)
- The application is desktop-only; no web or mobile interface
- All seven teams must share the same MySQL database instance on localhost
- Only one class in the entire application may extend Application — not applicable here as Swing is used

3. Functional Requirements

3.1 User Registration (FR-01)

3.1.1 Description

Any person with a valid GECT college email must be able to create an account. The registration form collects personal information, validates all inputs, checks email uniqueness, creates the appropriate user object (Student or Staff), and stores the record in the database.

3.1.2 Input Fields

Field	Type	Required	Validation Rule
Full Name	Text	Yes	Must not be empty
College Email	Email	Yes	Must end with @gectcr.ac.in, must be unique
Department	Dropdown	Yes	One of: CS, ECE, ME, CE, EEE
Role	Dropdown	Yes	STUDENT or STAFF
Roll Number	Text	If Student	Visible only when person_type = STUDENT
Employee ID	Text	If Staff	Visible only when person_type = STAFF
Mobile Number	Text	Yes	Must be 10 digits, start with 6-9
Password	Password	Yes	Minimum 6 characters
Confirm Password	Password	Yes	Must match Password field

3.1.3 Process Flow

1. User fills in all fields in the Registration Form
2. System validates all fields client-side before submission
3. System checks email uniqueness against the users table via UserDAO.emailExists()
4. System creates a Student or Staff object based on selected role
5. System calls AuthService.register() which calls UserDAO.registerUser()
6. On success, OTPService.sendOTP() generates a 6-digit code and saves it to otp_records
7. System navigates to OTP Verification Screen showing the generated OTP in a dialog

3.2 Login Authentication (FR-02)

3.2.1 Description

Registered users log in using their college email and password. The system validates credentials, starts a session by storing the User object in SessionManager, updates last_seen in the database, and navigates to the Dashboard.

3.2.2 Input Fields

Field	Type	Required	Validation
College Email	Text	Yes	Must match @gectcr.ac.in format
Password	Password	Yes	Must not be empty

3.2.3 Process Flow

8. User enters email and password on Login Screen
9. System validates field formats before making a DB call
10. AuthService.login() queries users table via UserDAO.getUserByEmail()
11. System compares entered password with stored password hash
12. On success: SessionManager.setCurrentUser(user) is called, last_seen updated, Dashboard opens
13. On failure: error message displayed inline on the Login Screen

3.3 OTP Verification (FR-03)

After registration, the user is shown the OTP Verification screen. A simulated 6-digit OTP is displayed in an information dialog (representing an email that would be sent in production). The user enters the OTP; the system verifies it against the stored value in otp_records and activates the account.

3.4 Forgot Password / Password Reset (FR-04)

A three-step card-based flow on a single screen:

- Step 1 — User enters their registered email. System validates the email exists and sends a simulated OTP.
- Step 2 — User enters the OTP. System verifies it against otp_records.
- Step 3 — User enters and confirms a new password. System calls AuthService.changePassword() which calls UserDAO.updatePassword().

3.5 Dashboard (FR-05)

After successful login, the Dashboard screen is shown. It displays the user's name, role, and department in a header. Six module buttons are displayed in a 3x2 grid — one for each of the other teams' modules. A Logout button clears the session and returns to the Login screen.

4. Non-Functional Requirements

ID	Category	Requirement
NF01	**Usability**	All screens must be navigable without any console interaction. All user input and output must be through GUI components only.
NF02	**Reliability**	The system must handle invalid inputs gracefully. No screen should crash on incorrect input; all errors must display as inline labels.
NF03	**Security**	Email must be validated for the college domain before any DB operation. Passwords must not be displayed in plaintext in any field.
NF04	**Maintainability**	Code must follow the 4-layer architecture (UI, Service, DAO, Model). Each method must have a Javadoc comment.
NF05	**Performance**	Login and registration operations must complete within 3 seconds under normal MySQL connectivity conditions.
NF06	**Portability**	The application must run on any Windows 10/11 machine with JDK 17 and MySQL 8.0 installed, without code changes.
NF07	**Integration**	Shared components (DBConnection, SessionManager, ScreenNavigator, User model) must be stable and not modified after Team 1 shares them.

5. Object-Oriented Design

5.1 OOP Concepts Used

OOP Concept	Where Applied	How It Is Used
Inheritance	User, Student, Staff	Student and Staff both extend the abstract User class. Common fields (email, fullName, department) are in User. Role-specific fields are in subclasses.
Encapsulation	All model classes	All fields in User, Student, and Staff are private. Accessed only through public getters and setters.
Abstraction	User class	User is an abstract class with the abstract method getDisplayInfo(). Concrete implementations are provided by Student and Staff.
Method Overriding	Student, Staff	Both Student and Staff override getDisplayInfo() to return role-appropriate display strings.
Constructor Overloading	User, Student, Staff	Each model class has two constructors: one for new registration (fewer fields) and one for loading from DB (all fields).
Singleton Pattern	SessionManager, DBConnection	SessionManager uses a private constructor and a static getInstance() method to ensure only one session exists throughout the app.
Polymorphism	UserDAO, AuthService	UserDAO.registerUser() accepts a User reference but uses instanceof to correctly handle Student vs Staff during INSERT.

5.2 Class Hierarchy

The module uses an inheritance hierarchy rooted at the User base class, with Student and Staff as derived classes. This reflects real-world domain modeling and satisfies the OOP requirement.

5.2.1 Class Diagram

The following tables represent the class diagram for the Authentication Module:

<<abstract>> User

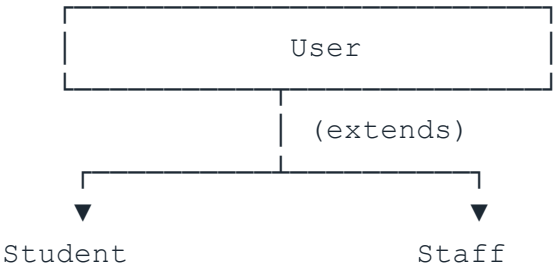
— Fields (private) —

```
- id: int
- email: String
- fullName: String
- password: String
- personType: String  // STUDENT | STAFF
- rollNoEmpId: String
- department: String
- mobile: String
```

```
- profilePic: String
- status: String
- isOnline: boolean
- lastSeen: Timestamp
- createdAt: Timestamp
```

— Methods (+) —

```
+ User()  // default constructor
+ User(id, email, fullName)  // partial constructor
+ User(id, email, fullName, password, personType, ...)  // full constructor
+ getters / setters for all fields
+ getDisplayRole(): String  // overridden in subclasses
+ getIdentifier(): String   // overridden in subclasses
+ countNullFields(): int
+ getCreatedDate(): String
```



Student (extends User)

```
- rollNo: String  (from base)
+ Student(id, email, fullName,
rollNo)
+ getDisplayRole(): "Student"
+ getIdentifier(): "Roll No: " +
rollNo
```

Staff (extends User)

```
- empId: String  (from base)
+ Staff(id, email, fullName, empId)
+ getDisplayRole(): "Staff"
+ getIdentifier(): "Emp ID: " +
empId
```

5.2.2 Supporting Classes

Class	Responsibility	Key Methods
AuthDAO	Database CRUD for authentication	authenticate(), register(), isEmailExists(), updatePassword()
AuthService	Business logic, validation, session handling	login(), register(), logout()
LoginPage	Swing login form panel	validateFields(), performLogin()
RegisterPage	Swing registration form with OTP	handleGenerateOtp(), handleRegister()
ForgotPasswordPage	Forgot password with OTP reset	handleGenerateOtp(), handleUpdatePassword()

AuthModulePanel	CardLayout container for all auth screens	Navigates LOGIN / REGISTER / FORGOT_PASSWORD
SessionManager	Singleton: stores logged-in user	login(), logout(), getCurrentUser()
PasswordUtils	SHA-256 password hashing/verification	hashPassword(), verifyPassword()
ValidationUtils	Inline static helpers for email/password	isValidEmail(), isValidPassword(), trim()
DBConnection	JDBC connection utility	getConnection()
EventBus	Pub-Sub for cross-module events	subscribe(), publish()

6. Data Flow Diagrams (DFD)

6.1 DFD Level 0 — Context Diagram

The Level 0 DFD shows the Authentication Module as a single process interacting with external entities.

DFD Level 0 — Context Diagram

```
                          [ User (Actor) ]
                                |
        Registration Data / Login Credentials / Password Reset Request
                                ▼
           +-----------------------------------------------+
           |   0.0  GECT Connect — Authentication System   |
           +-----------------------------------------------+
                        |                          |
             Auth Results / Session      SQL Queries / Responses
                        ▼                          ▼
           [ Other Modules ]           [ MySQL Database (D1) ]
```

6.2 DFD Level 1 — Authentication Module

Level 1 expands the single process into the four primary sub-processes of the Authentication Module.

DFD Level 1 — Authentication Module Sub-Processes

```
[ User ]
   |-- Registration Data ------>  [1.1 Validate & Register User]
   |                                        |-- Valid Data ------> [D1:
users]
   |                                        |<- Dup Check -------- [D1:
users]
   |                                        |-- OTP -----------> User Screen
   |                                        |<- OTP Entry ------- [User]
   |                                        |-- Success/Fail --> [User]

   |-- Login Credentials -------> [1.2 Authenticate User]
   |                                        |-- Email Query ----> [D1: users]
   |                                        |<- User Record ------ [D1:
users]
   |                                        |-- Auth Token ----->
SessionManager
   |                                        |-- Dashboard Data --> [User]

   |-- Email + OTP Request -----> [1.3 Reset Password]
   |                                        |-- Email Check ----> [D1: users]
   |                                        |-- OTP -----------> User Screen
   |                                        |-- New Password --> [D1: users]

   |-- Logout Request -----------> [1.4 Manage Session]
   |                                        |-- Online=false --> [D1: users]
   |                                        |-- Event Publish -> EventBus
   |                                        |-- Navigate ------> Login Screen
```

6.3 DFD Data Dictionary

Data Flow	From → To	Contents
Registration Data	User → 1.1	fullName, email, role, rollNoEmpId, mobile, password, confirmPassword
OTP Display	1.1 → User Screen	6-digit integer (simulated, shown on screen)
OTP Entry	User → 1.1	6-digit string entered by user
Login Credentials	User → 1.2	email, password
Auth Token	1.2 → SessionManager	User object (id, email, fullName, personType, rollNoEmpId)
Dashboard Data	1.2 → User	Full name, department, role, welcome message
Reset Request	User → 1.3	email
New Password	1.3 → D1	Hashed new password
Online Status	1.4 → D1	is_online=false, last_seen=NOW()

7. Database Schema

7.1 Entity-Relationship Overview

The Authentication Module primarily owns the users table and interacts with the settings table. The contacts, messages, groups, and notifications tables are owned by downstream modules but reference users.id as a foreign key.

7.2 Primary Table: users

Column	Data Type	Constraint	NN	Description
id	INT	PK, AUTO_INCREMENT	Y	Unique user identifier
email	VARCHAR(255)	UNIQUE	Y	College email — must match *@gectcr.ac.in
full_name	VARCHAR(255)	—	Y	User's full name
password	VARCHAR(255)	—	Y	SHA-256 hashed password
person_type	ENUM('STUDENT','STAFF')	—	Y	Role discriminator
roll_no_emp_id	VARCHAR(50)	UNIQUE	Y	Roll number (Student) or Employee ID (Staff)
department	VARCHAR(100)	—	N	Academic or administrative department
mobile	VARCHAR(15)	—	N	Contact number
profile_pic	VARCHAR(500)	—	N	File path to profile picture
status	VARCHAR(255)	DEFAULT '...'	N	User status message
is_online	BOOLEAN	DEFAULT FALSE	N	Real-time online status
last_seen	TIMESTAMP	NULL	N	Last activity timestamp
created_at	TIMESTAMP	DEFAULT NOW()	N	Account creation time

7.3 Supporting Tables (Auth-Adjacent)

The following tables are accessed or updated during authentication flows:

7.3.1 settings

Column	Type	Constraint	Description
user_id	INT	PK, FK → users(id)	Links settings to user
theme	ENUM('LIGHT','DARK')	DEFAULT 'LIGHT'	UI theme preference
privacy	ENUM(...)	DEFAULT 'PUBLIC'	Profile visibility
notifications_enabled	BOOLEAN	DEFAULT TRUE	Push notification toggle

7.4 Normalization Analysis

The schema has been designed to satisfy Third Normal Form (3NF). The analysis below is from the perspective of Team 1's primary table: users.

7.4.1 First Normal Form (1NF) ✓

- All columns contain atomic (indivisible) values — no multi-valued or composite attributes.
- Each row is uniquely identified by the primary key id.
- Column names are unique within the table.
- Row order does not carry information.

7.4.2 Second Normal Form (2NF) ✓

- The primary key is a single column (id — surrogate key), so there are no partial dependencies possible.
- All non-key attributes (email, full_name, password, …) are fully functionally dependent on id.
- No partial dependency exists.

7.4.3 Third Normal Form (3NF) ✓

- There are no transitive dependencies: no non-key column determines another non-key column.
- person_type acts as a role discriminator but does not cause transitive dependency — Student and Staff properties are stored in a single users table with the shared rollNoEmpId column, with meaning determined by person_type at the application layer.
- department is directly dependent on the user (id), not on another non-key attribute.
- The settings table was extracted to avoid transitive dependency — theme, privacy, etc., depend on user_id, not on any other users field.

Conclusion: The schema attains 3NF. Denormalization has not been applied; the contacts helper table is an intentional performance optimization (caching last_message_id and unread_count) and is documented as such.

8. SQL Script — Authentication Module

8.1 Schema Creation

```sql
-- ====================================================
-- GECT Connect — Authentication Module DB Schema
-- Team 1 | GEC Thrissur
-- ====================================================
CREATE DATABASE IF NOT EXISTS gect_connect
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE gect_connect;

-- 1. USERS TABLE (Team 1 — Primary Owner)
CREATE TABLE IF NOT EXISTS users (
    id              INT PRIMARY KEY AUTO_INCREMENT,
    email           VARCHAR(255) UNIQUE NOT NULL,
    full_name       VARCHAR(255) NOT NULL,
    password        VARCHAR(255) NOT NULL,
    person_type     ENUM('STUDENT','STAFF') NOT NULL,
    roll_no_emp_id  VARCHAR(50) UNIQUE NOT NULL,
    department      VARCHAR(100),
    mobile          VARCHAR(15),
    profile_pic     VARCHAR(500),
    status          VARCHAR(255) DEFAULT 'Hey! I am on GECT Connect.',
    is_online       BOOLEAN DEFAULT FALSE,
    last_seen       TIMESTAMP NULL,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_email CHECK (email LIKE '%@gectcr.ac.in')
);

-- 2. SETTINGS TABLE
CREATE TABLE IF NOT EXISTS settings (
    user_id              INT PRIMARY KEY,
    theme                ENUM('LIGHT','DARK') DEFAULT 'LIGHT',
    privacy              ENUM('PUBLIC','PRIVATE','CONTACTS') DEFAULT 'PUBLIC',
    notifications_enabled BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- 3. TEST / SEED DATA
INSERT INTO users (email, full_name, password, person_type, roll_no_emp_id)
VALUES
  ('admin@gectcr.ac.in','System Admin','password123','STAFF','STAFF001'),
  ('student1@gectcr.ac.in','John Doe','student123','STUDENT','STU001');
```

9. GUI Screens Description

9.1 Screen Inventory

Screen	Card Key	Description
Login Screen	LOGIN	Email + password input. Real-time validation, SwingWorker async call. Toggle password visibility.
Registration Screen	REGISTER	Full registration form with role dropdown (Student/Staff), OTP generation button, OTP panel revealed after generation.
OTP Verification	(inline on REGISTER)	Integrated into Register and Forgot Password screens. OTP displayed on-screen (simulation). Real-time match check enables the submit button.
Forgot Password	FORGOT_PASSWORD	Email input → OTP → new password + confirm. Password reset on OTP match.
Dashboard	HOME (LandingPage)	Post-login landing page showing user name, role, department. Navigation stubs to all other modules. Logout button.

9.2 Navigation Flow

```
                    Application Startup
                            |
                            ▼
                    [LOGIN SCREEN]
          |             |             |
     Login OK     Register    Forgot Password
          |             |             |
          ▼             ▼             ▼
[DASHBOARD]    [REGISTER]  [FORGOT_PASSWORD]
          |             |             |
     Logout      OTP Gen → OTP Verify → Login
                        |
                        ▼
                    [LOGIN SCREEN]
```

10. Test Log

10.1 Test Cases — Standard Format

TC#	Test Case	Input	Expected Output	Status	Screen
TC01	Valid Registration	name=John Doe, email=john@gectcr.ac.in, role=STUDENT, roll=STU001, pass=pass123	OTP displayed; on correct OTP — success toast, redirect to Login	**PASS**	Register
TC02	Invalid Email Domain	email=john@gmail.com	Error: 'Use your college email'	**PASS**	Register
TC03	Short Password	password=abc	Error: 'Min 6 characters'	**PASS**	Register
TC04	Password Mismatch	pass=pass123, confirm=pass456	Error: 'Passwords do not match'	**PASS**	Register
TC05	Duplicate Email	Already-registered email	Error: 'Email already registered'	**PASS**	Register
TC06	Wrong OTP	OTP=111111 (incorrect)	Register button stays disabled	**PASS**	Register
TC07	Correct OTP	OTP matches displayed value	Register button enabled, successful DB insert	**PASS**	Register
TC08	Valid Login	email=john@gectcr.ac.in, pass=pass123	Session created, Dashboard displayed	**PASS**	Login
TC09	Wrong Password	Correct email, wrong password	Error: 'Invalid password'	**PASS**	Login
TC10	Unknown Email	email not in DB	Error: 'Account not found'	**PASS**	Login
TC11	Forgot Password — Valid Flow	Registered email, correct OTP, new password ≥ 6	Password updated, redirect to Login	**PASS**	ForgotPwd
TC12	Forgot Password — Unknown Email	email not in DB	Error: 'No account found'	**PASS**	ForgotPwd
TC13	Logout	Click Logout on Dashboard	Session cleared, redirect to Login	**PASS**	Dashboard
TC14	Role Field Swap	Change role dropdown Student → Staff	Placeholder changes from 'Roll No' to 'Employee ID'	**PASS**	Register
TC15	Student subtype login	Login as STUDENT account	getDisplayRole() returns 'Student', Dashboard shows Roll No	**PASS**	Dashboard

11. Module Integration — Team 1 Perspective

11.1 Events Published by Team 1

Event Name	Payload	Consumed by
AUTH_LOGIN_SUCCESS	User object	MainFrame, LandingPage, ProfileModulePanel, NotificationModulePanel, all modules via INIT_MODULES_AFTER_LOGIN
INIT_MODULES_AFTER_LOGIN	User object	All 6 other team panels — triggers data load after login
AUTH_REGISTER_SUCCESS	email: String	(Informational — for analytics if needed)
GLOBAL_LOGOUT	null	MainFrame — resets to Login screen; clears all module state

11.2 Data Provided to Other Teams

Consuming Team	Data Used from users Table	How Accessed
Team 2 — Profile	id, email, full_name, department, profile_pic, status, roll_no_emp_id, mobile, created_at	SessionManager.getCurrentUser() + ProfileDAO query
Team 3 — Contacts	id, email, full_name, department, profile_pic	ContactsDAO joins users table using user_id FK
Team 4 — Chat	id (sender_id, receiver_id in messages table)	SessionManager.getCurrentUserId()
Team 5 — Groups	id (creator_id, user_id in group_members)	SessionManager.getCurrentUserId()
Team 6 — Notifications	id (user_id in notifications table)	SessionManager.getCurrentUserId()
Team 7 — Settings	id (user_id in settings table)	SessionManager.getCurrentUserId()